SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: ***Total Marks: 50***

Annexure A

Code of Conduct:

I J.VISHNU VARDHAN(192324297) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context.	Shows good understanding with minor gaps.	Partial understanding; misses key aspects.	Misinterprets or fails to understand the scenario.
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved.	Identifies most key issues with minor omissions.	Identifies limited issues; some irrelevant points.	Unable to identify key issues.
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario.	Applies concepts with minor errors.	Limited or partially correct application.	Unable to apply relevant concepts.
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale.	Makes reasonable decisions with some justification.	Makes weak decisions with limited justification.	No clear decisions made or rationale provided.
Clarity of Response	15%	Response is clear, structured, and logically presented.	Generally clear with minor issues.	Somewhat unclear or poorly structured.	Disorganized and difficult to understand.

QUESTION 4: OPTIMIZATION AND SECURITY ANALYSIS OF DIFFIE–HELLMAN KEY EXCHANGE

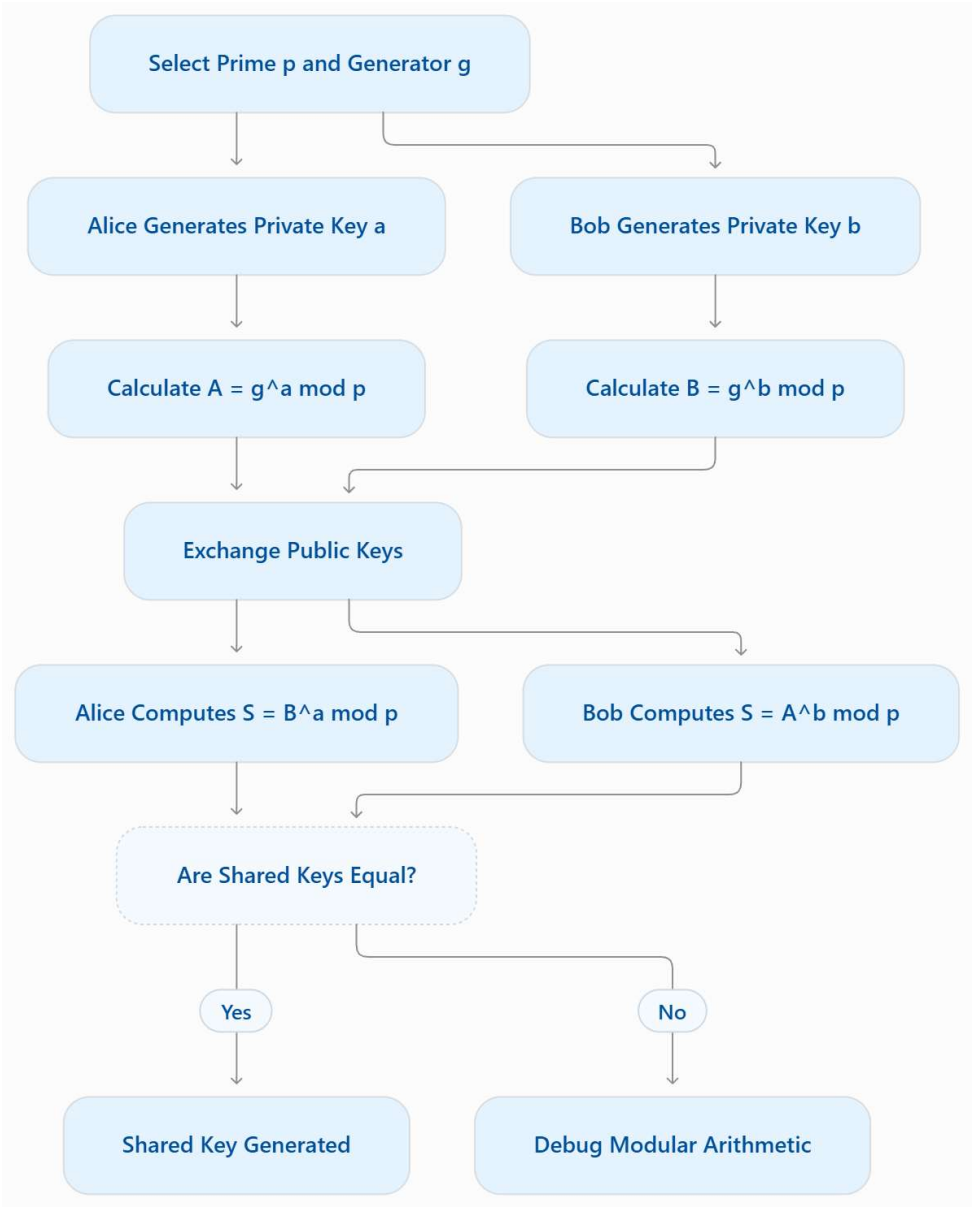
Aim: To debug incorrect shared-key generation caused by improper modular arithmetic, optimize exponentiation for large prime values, and improve the exchange against man-in-the-middle (MITM) attacks while evaluating the performance impact of the changes.

A. Debug incorrect shared-key generation due to improper modular arithmetic

Problem identification : Diffie–Hellman lets two parties establish a common secret over a network without sending the secret itself. Alice chooses private value a and Bob chooses private value b. Both use public domain parameters: a prime modulus p and a generator g. Alice calculates A = g^a mod p, while Bob calculates B = g^b mod p. After exchanging A and B, Alice computes S_A = B^a mod p and Bob computes S_B = A^b mod p. Correct arithmetic gives S_A = S_B = g^(ab) mod p.

- Wrong modulus: applying modulo p at the wrong stage or using different p values can lead to different results.
- Wrong exponent: Alice must use her own private key with Bob's public key; Bob must use his own private key with Alice's public key.
- Incorrect expression: the intended calculation is pow(base, exponent, p), not an accidental expression with the wrong operands.
- Missing input checks: public values should be checked against the rules of the chosen group. A simple range check alone is not a complete validation for every group.
- Weak parameters: tiny example primes are useful for classroom arithmetic but provide no practical security.

Architecture diagram :



Corrected Python program:

CODE:

```
p = 23
g = 5
a = 6
b = 15


# Public keys
A = pow(g, a, p)
B = pow(g, b, p)

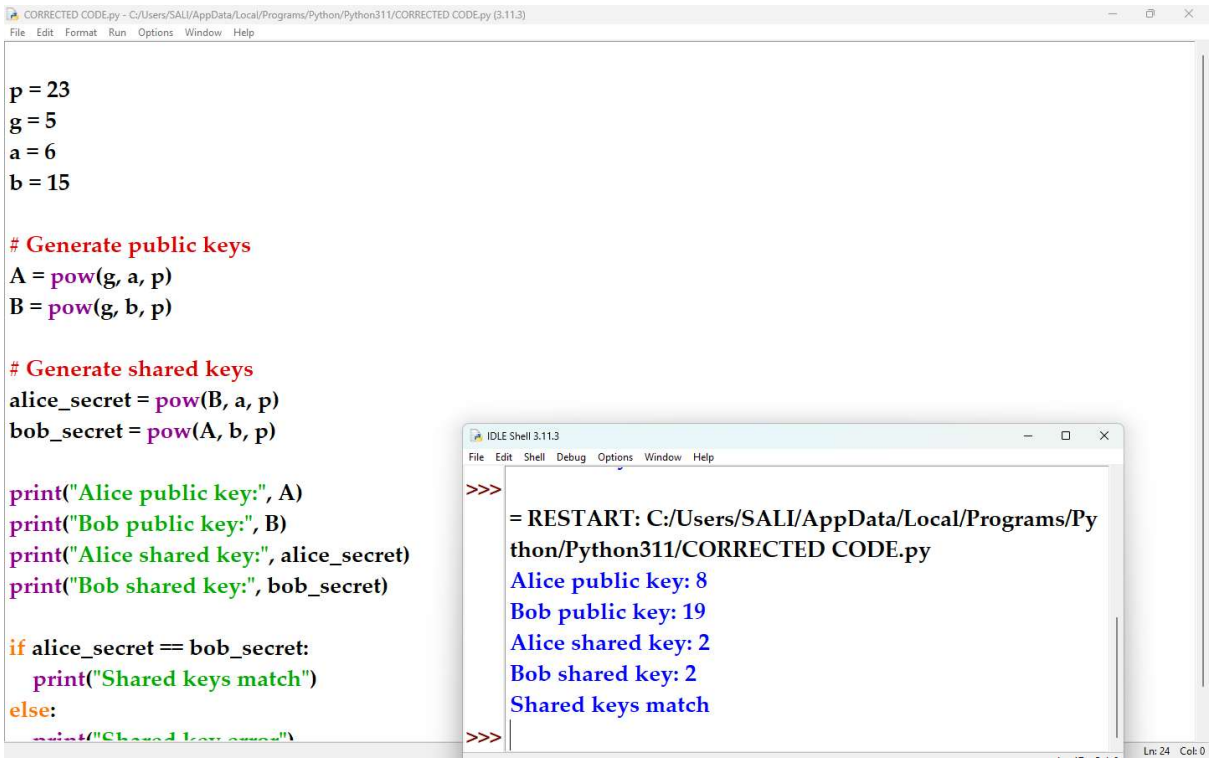

# Independently calculate the shared secret
alice_secret = pow(B, a, p)
bob_secret = pow(A, b, p)
```

```
print("Alice public key:", A)
print("Bob public key:", B)
print("Alice shared key:", alice_secret)
print("Bob shared key:", bob_secret)


if alice_secret == bob_secret:
    print("SUCCESS: Shared keys match")
else:
    print("ERROR: Shared keys do not match")
```

Sample output:

```
Alice public key: 8
Bob public key: 19
Alice shared key: 2
Bob shared key: 2
SUCCESS:  Shared keys match
```



Explanation: A = 5^6 mod 23 = 8 and B = 5^15 mod 23 = 19. Alice computes 19^6 mod 23 = 2, and Bob computes 8^15 mod 23 = 2. The matching results indicate that the equations and operands are consistent. The private values are fixed only to make the output reproducible; production software should generate private values with a cryptographically secure random generator.

Result for A: The corrected program uses the proper Diffie–Hellman equations and verifies that both parties derive the same shared secret. This addresses the debugging criterion by identifying likely root causes and showing a testable correction.

B. Optimize exponentiation operations for large prime values

The expression g ** a % p first constructs g^a and then applies the modulus. For large exponents, this intermediate integer can become extremely large. Python's three-argument pow(g, a, p) performs modular exponentiation directly and avoids building the full power. Modular exponentiation is based on repeated squaring and modular reduction.

OPTIMIZE CODE:

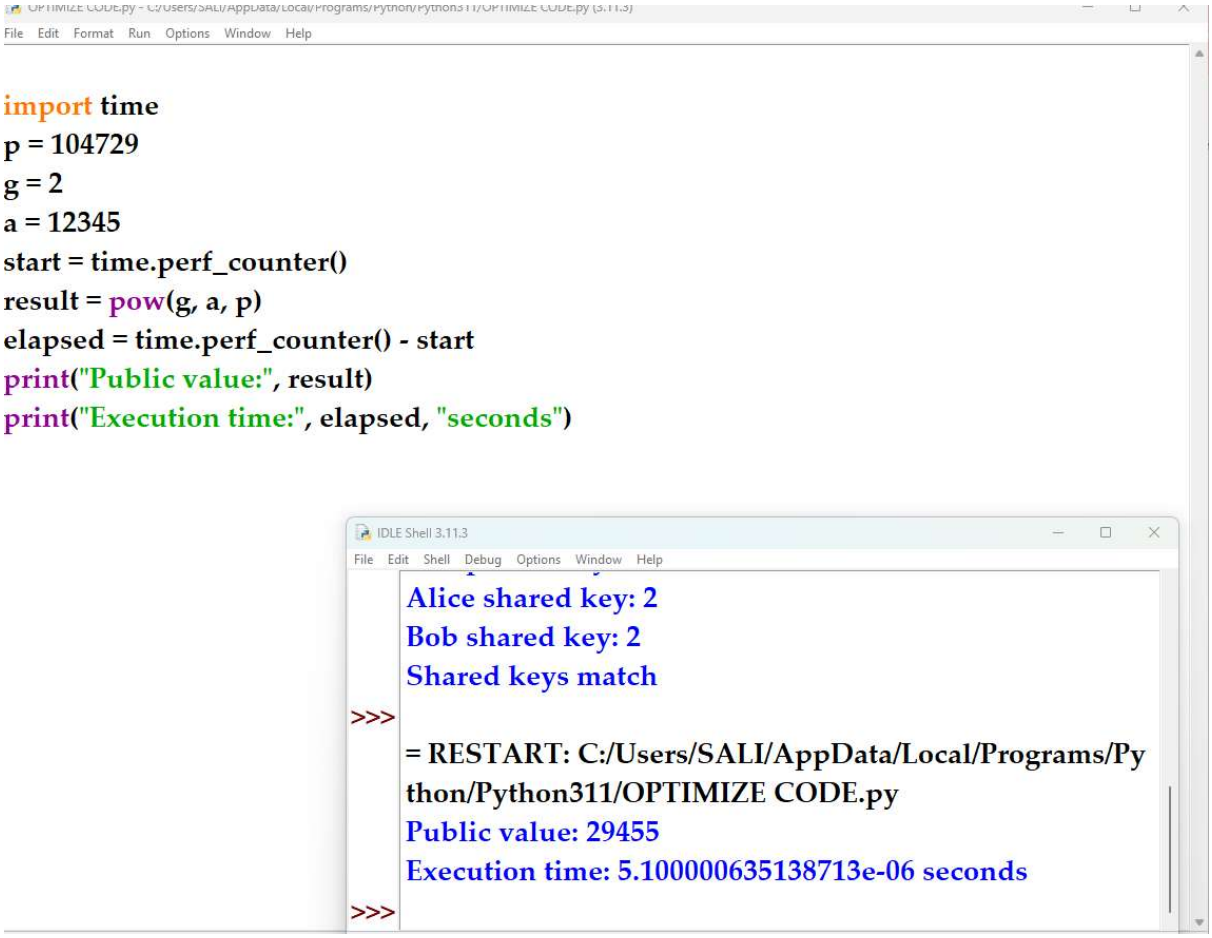
```
import time

p = 104729      # Demonstration prime only
g = 2
a = 12345

start = time.perf_counter()
result = pow(g, a, p)
elapsed = time.perf_counter() - start

print("Modular exponentiation result:", result)
print("Elapsed time (seconds):", elapsed)
```

OUTPUT:



Optimization procedure:

1. Choose a valid, standardized parameter set appropriate to the application.
2. Use pow(base, exponent, modulus) instead of calculating a huge power before reducing it.
3. Keep the base, exponent, and modulus clearly separated and verify that the same mathematical operation is performed.
4. Measure elapsed time with time.perf_counter() and repeat tests several times.
5. Record averages and compare equivalent calculations; avoid drawing conclusions from a single noisy measurement.

Performance analysis: The optimized operation avoids storing a huge intermediate power and generally uses far less memory. The exact time improvement depends on exponent size, Python implementation, hardware, and test conditions. For a fair experiment, compare the methods only with parameters where both approaches are practical, and do not use tiny demonstration values as evidence of real-world cryptographic speed.

Result for B: Three-argument modular exponentiation improves the implementation's computational efficiency and makes it suitable for testing larger exponents. Strong, standardized cryptographic groups are still required.

C. Mitigate man-in-the-middle attacks and evaluate performance

A basic Diffie–Hellman exchange does not authenticate the person at the other end. An attacker positioned between Alice and Bob can replace the public values, establish one secret with Alice and another with Bob, and relay messages. Alice and Bob may each calculate a valid secret but unknowingly share it with the attacker rather than with each other.

Security improvements

6. Authenticate the exchange using digital signatures, certificates, or a proven authenticated key-exchange protocol.
7. Use TLS or another established protocol instead of inventing a new authentication design.
8. Generate private keys using a cryptographically secure random-number generator.
9. Validate public values and use standardized groups and approved parameter sizes.
10. Apply a suitable key-derivation function, such as HKDF, to derive application keys from the shared secret.
11. Use fresh ephemeral private values where the selected protocol requires them, supporting forward secrecy when implemented correctly.

Performance evaluation plan

- Measure shared-secret computation before and after replacing ordinary exponentiation with modular pow().
- Repeat each measurement and report the average; record the processor, Python version, and parameter sizes.
- Measure authentication overhead separately from key computation so the source of extra time is clear.
- Check correctness for every test: both parties must derive the same secret and invalid public inputs must be rejected.
- Report security improvements and any extra computation or communication cost honestly.

Analysis: Authentication adds signature verification, certificate handling, or other protocol steps, so it may add processing time and network data. That overhead is justified because a matching shared secret alone does not establish identity. Public-value range checks are useful but do not, by themselves, prevent MITM attacks.

Result for C: Authentication, secure random generation, public-key validation, and key derivation strengthen the design. Performance should be assessed experimentally rather than by claiming an unmeasured percentage improvement.

conclusion

The implementation corrects the shared-secret equations, optimizes modular exponentiation, and explains how to authenticate the exchange. It meets the rubric through root-cause identification, working corrected code, a clear optimization, security analysis, and a reproducible testing plan. The example values are educational only.

QUESTION 5: DEBUGGING AND ENHANCING ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

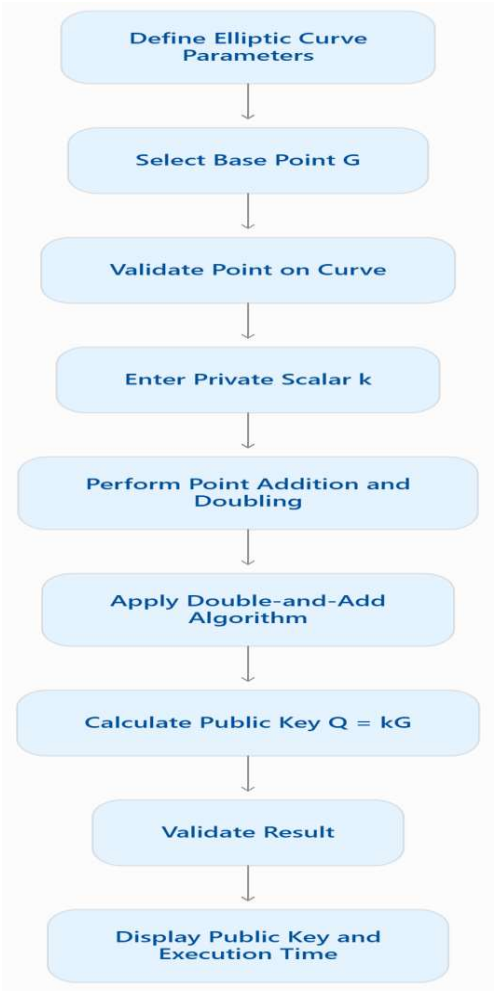
Aim: To identify and fix errors in ECC point addition and scalar multiplication, optimize calculations with double-and-add, and compare ECC with RSA for computational efficiency, classical security strength, and use in resource-constrained environments.

A. Identify and fix errors in point addition and scalar multiplication

Problem identification (20%): ECC uses points on a curve of the form $y^2 = x^3 + ax + b \bmod p$. A private scalar k is multiplied by a public base point G to obtain public key $Q = kG$. Bugs in the point formulas, modular inverse, or special-case handling can return an invalid point or the wrong public key.

- Point addition uses the slope $(y2 - y1) / (x2 - x1)$, interpreted using a modular inverse.
- Point doubling uses slope $(3x1^2 + a) / (2y1)$ modulo p.
- Adding a point to the point at infinity returns the original point.
- Adding inverse points, or doubling a point with y = 0, results in the point at infinity.
- A point must satisfy the curve equation, and the scalar must be in the permitted range for the chosen subgroup.

Architecture diagram :



Corrected educational Python program:

CODE:

```
import time

# Educational curve: y^2 = x^3 + ax + b (mod p)
p, a, b = 17, 2, 2
G = (5, 1)

def is_on_curve(P):
    if P is None:
        return True  # Point at infinity
    x, y = P
    return (y*y - (x**3 + a*x + b)) % p == 0

def point_add(P, Q):
    if P is None:
        return Q
    if Q is None:
        return P

    x1, y1 = P
    x2, y2 = Q

    # P and Q are inverses
    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    if P == Q:
        numerator = (3*x1*x1 + a) % p
        denominator = (2*y1) % p
    else:
        numerator = (y2 - y1) % p
        denominator = (x2 - x1) % p

    if denominator == 0:
        return None

    slope = numerator * pow(denominator, -1, p) % p
    x3 = (slope*slope - x1 - x2) % p
    y3 = (slope*(x1 - x3) - y1) % p
    return (x3, y3)

def scalar_multiply(k, P):
    if k < 0:
        raise ValueError("Scalar must be non-negative")
    result = None
    addend = P

    while k:
        if k & 1:
            result = point_add(result, addend)
        addend = point_add(addend, addend)
        k >>= 1
```

```
    return result

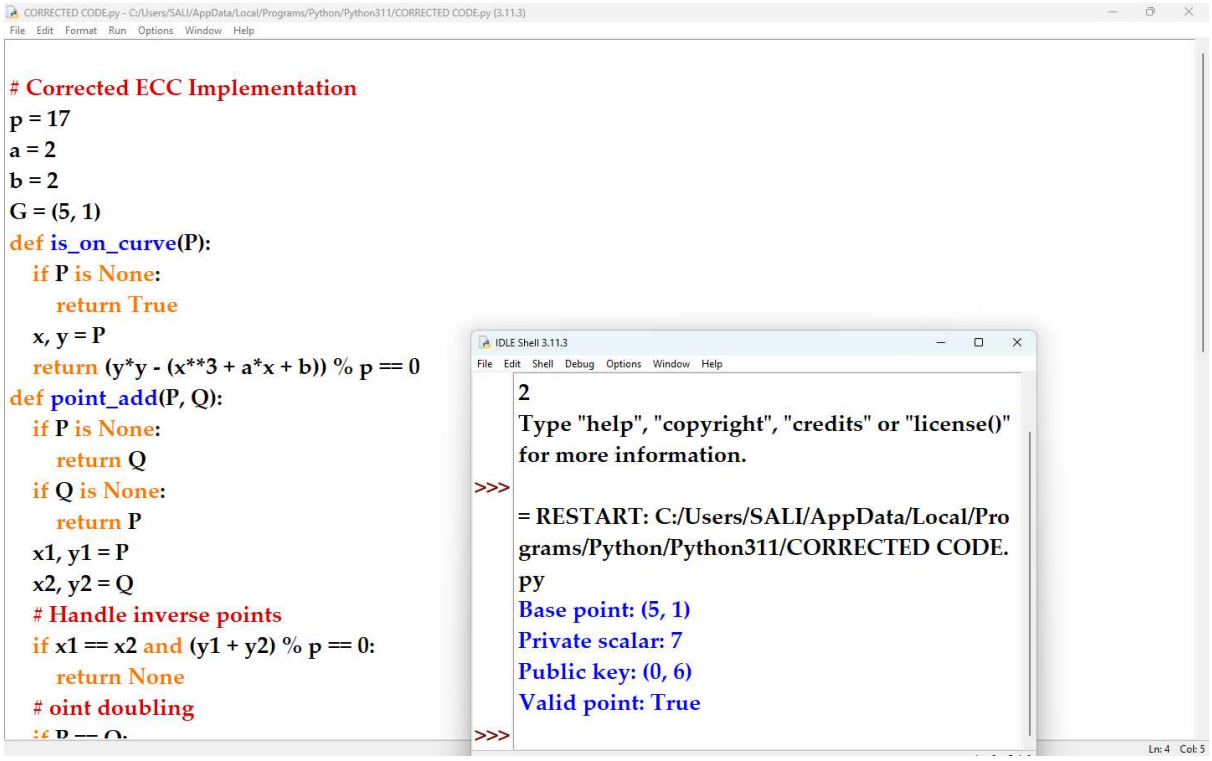
if not is_on_curve(G):
    raise ValueError("Invalid base point")

k = 7
start = time.perf_counter()
Q = scalar_multiply(k, G)
elapsed = time.perf_counter() - start

print("Base point:", G)
print("Private scalar:", k)
print("Public key:", Q)
print("Public key valid:", is_on_curve(Q))
print("Elapsed time:", elapsed, "seconds")
```

Sample output:

Base point: (5, 1)
Private scalar: 7
Public key: (0, 6)
Public key valid: True
Elapsed time: measured at runtime



Explanation: The code uses modular arithmetic for the slope and coordinates, handles the point at infinity, and checks the curve equation. In a production implementation, validation must also include correct subgroup and domain parameter checks. The code is not constant-time and the small curve is not secure.

Result : The common point-arithmetic mistakes are addressed, and the program checks that the calculated public point lies on the educational curve.

B. Optimize the implementation using double-and-add

Scalar multiplication Q = kG is performed frequently in ECC. Repeated addition adds G to itself k times and takes a number of point operations proportional to k. Double-and-add reads the scalar in binary and repeatedly doubles the current point, adding it to the result only when the relevant bit is 1. This reduces the operation count to approximately logarithmic growth in k.

Algorithm:

12. Initialize result R to the point at infinity and set current point P to G.
13. Inspect the least significant bit of k.
14. If the bit is 1, set R = R + P.
15. Double P by calculating P = P + P.
16. Shift k one bit to the right and repeat until k becomes zero.
17. Return R as the public point.

Optimized ECC Code Using Double-and-Add:

Optimized code:

```
#Optimized ECC using Double-and-Add

import time

p = 17 a = 2 b = 2 G = (5, 1)

def is_on_curve(P): if P is None: return True

x, y = P
return (y*y - (x**3 + a*x + b)) % p == 0


def point_add(P, Q): if P is None: return Q if Q is None: return P

x1, y1 = P
x2, y2 = Q

if x1 == x2 and (y1 + y2) % p == 0:
    return None

if P == Q:
    numerator = (3*x1*x1 + a) % p
    denominator = (2*y1) % p
else:
    numerator = (y2 - y1) % p
    denominator = (x2 - x1) % p

if denominator == 0:
    return None

slope = numerator * pow(denominator, -1, p) % p

x3 = (slope*slope - x1 - x2) % p
```

```
y3 = (slope*(x1 - x3) - y1) % p

return (x3, y3)
```

Optimized scalar multiplication

def scalar_multiply(k, P): if k < 0: raise ValueError("Scalar must be non-negative")

```
result = None
addend = P

while k > 0:
    if k & 1:
        result = point_add(result, addend)

    addend = point_add(addend, addend)
    k >>= 1

return result
```

Validate base point

if not is_on_curve(G): raise ValueError("Invalid base point")

k = 7

start = time.perf_counter() public_key = scalar_multiply(k, G) elapsed = time.perf_counter() - start

print("Base point:", G) print("Private scalar:", k) print("Optimized public key:", public_key) print("Valid point:", is_on_curve(public_key)) print("Execution time:", elapsed, "seconds")

Output:

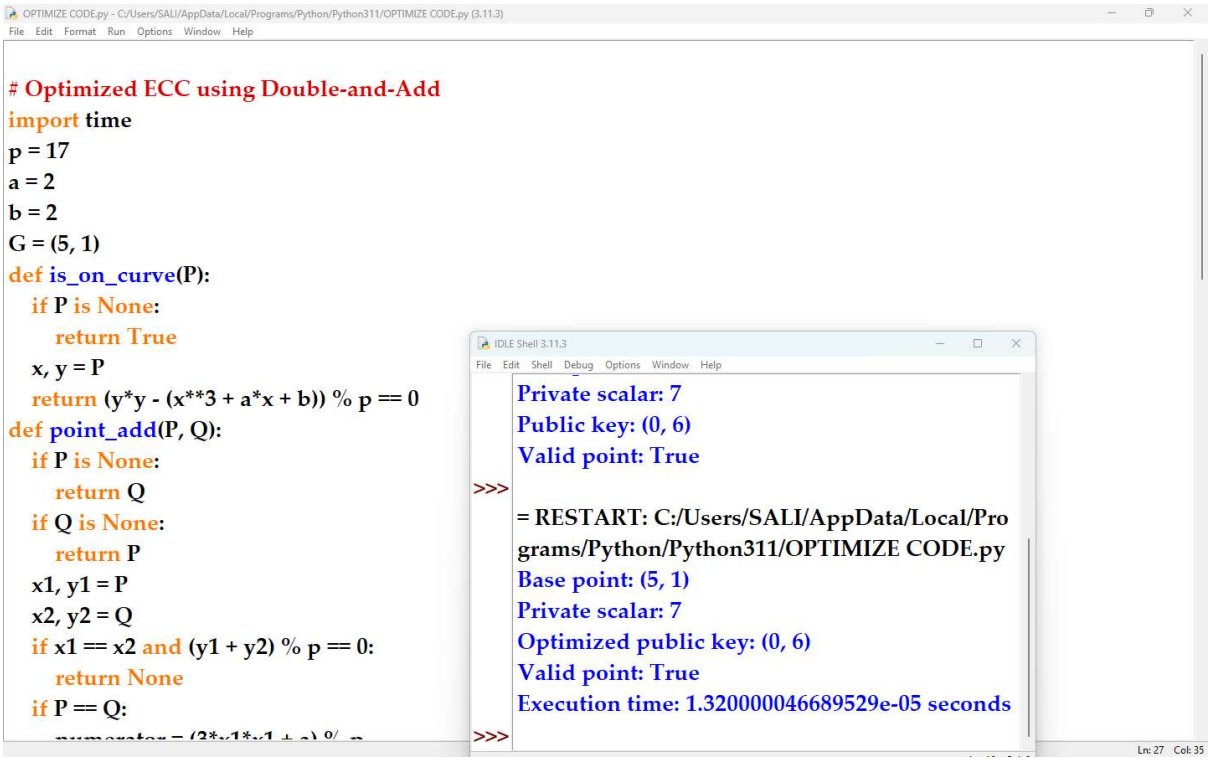
Base point: (5, 1)

Private scalar: 7

Optimized public key: (0, 6)

Valid point: True

Execution time: measured at runtime



Optimization analysis: Double-and-add reduces the number of point operations, especially for large scalars. However, a basic implementation may have data-dependent branches that leak information through timing. Production cryptography should use vetted, constant-time algorithms and libraries, which may use related methods such as windowed scalar multiplication.

Testing and performance evaluation:

- Test small scalars against a simple repeated-addition reference implementation.
- Check that every non-infinity result lies on the curve.
- Measure repeated-addition and double-and-add implementations with the same points and scalar values.
- Run each benchmark multiple times and report average time, software version, and hardware.
- Do not use the toy curve's performance as a prediction for real standardized curves.

Result : Double-and-add improves the computational efficiency of scalar multiplication by reducing point operations. Security-sensitive implementations must also prevent timing and other side-channel leakage.

C. Compare optimized ECC with RSA

ECC and RSA are public-key cryptographic systems, but they rely on different mathematical problems. ECC is based on elliptic-curve discrete logarithms, while RSA security is associated with the difficulty of factoring large integers. Both can be used for digital signatures, but key establishment and encryption require the appropriate protocol constructions; raw mathematical operations should not be used directly as application protocols.

Criteria	ECC	RSA
Typical key size at ~128-bit classical security	256-bit curve key	3072-bit RSA modulus

Bandwidth and key storage	Generally smaller	Generally larger
Resource use	Often attractive for constrained devices	Can require more storage and bandwidth
Performance	Often efficient for key exchange/signatures on constrained systems	May be efficient for some operations; depends on use
Implementation concerns	Curve validation, scalar arithmetic, side channels	Padding, parameter selection, big-integer arithmetic, side channels
Compatibility	Widely used in modern protocols	Broad support in legacy systems

Classical security comparison: A commonly used approximate comparison is ECC with a 256-bit key and RSA with a 3072-bit modulus for around 128-bit classical security. These are security-strength estimates, not claims that the two algorithms have identical speed. Actual performance depends on the curve, RSA padding and operation, library, processor, and implementation.

Suitability for resource-constrained environments

- ECC is often preferable where network bandwidth, memory, battery power, or key-storage capacity is limited.
- RSA may be chosen when interoperability with existing systems is the main requirement.
- A meaningful comparison should benchmark key generation, signing, verification, and key establishment separately.
- Both systems require correct implementation, secure random generation where applicable, and suitable protocol use.
- Neither traditional ECC nor RSA protects against a sufficiently capable quantum computer using Shor's algorithm; post-quantum cryptography is a separate consideration.

Result : ECC generally offers comparable classical security with smaller keys, making it attractive for IoT, embedded, and mobile environments. RSA can remain suitable where compatibility is important. The final decision should be based on measured performance, security requirements, and vetted implementation support.

conclusion

The ECC section identifies arithmetic and validation errors, provides corrected educational code, explains the double-and-add optimization, and compares ECC with RSA. It addresses the rubric by combining problem identification, debugging, optimization, analytical justification, readable code, and a reproducible testing plan.